Seth Lunas - 033183876
Han Htoo Zin - 033310860
Christopher Dominguez - 033228505

CECS 327 - Project 1

Dockerfile Steps Explanation:

python_app/Dockerfile

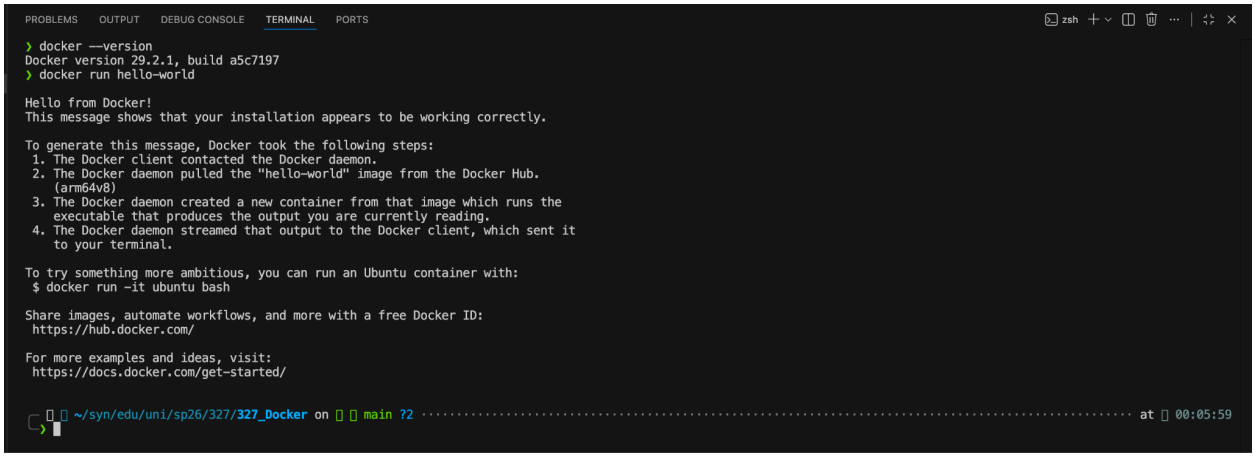
- FROM python:3.11 - Uses Python 3.11 base image with Debian Linux
- WORKDIR /app - Sets working directory to /app inside container
- COPY app.py . - Copies Python script from host to container
- CMD ["python3", "app.py"] - Runs the script when container starts

server-clients/Dockerfile

- FROM python:3.11 - Python 3.11 base image
- WORKDIR /app - Sets /app as working directory
- COPY server.py . / COPY client.py . - Copies both scripts into container
- EXPOSE 5000 - Documents that port 5000 is used
- CMD ["python", "-u", "server.py"] - Runs server with unbuffered output (-u for real-time logs)

Screenshots:

2. Install Docker



3. Run and explore first container

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                                    zsh

❯ docker run -it alpine:latest sh
/ # ls
bin    dev    etc    home   lib    media  mnt    opt    proc   root   run    sbin   srv    sys    tmp    usr    var
/ # echo "Hello Docker!" > test.txt
/ # ls
bin       etc       lib       mnt       proc      run       srv       test.txt  usr
dev       home      media     opt       root      sbin      sys       tmp       var
/ # cat test.txt
Hello Docker!
/ # exit

~/syn/edu/uni/sp26/327/327_Docker on main ?2 ····· took 30s at 00:08:36
❯
```

4. Creating a custom Docker image

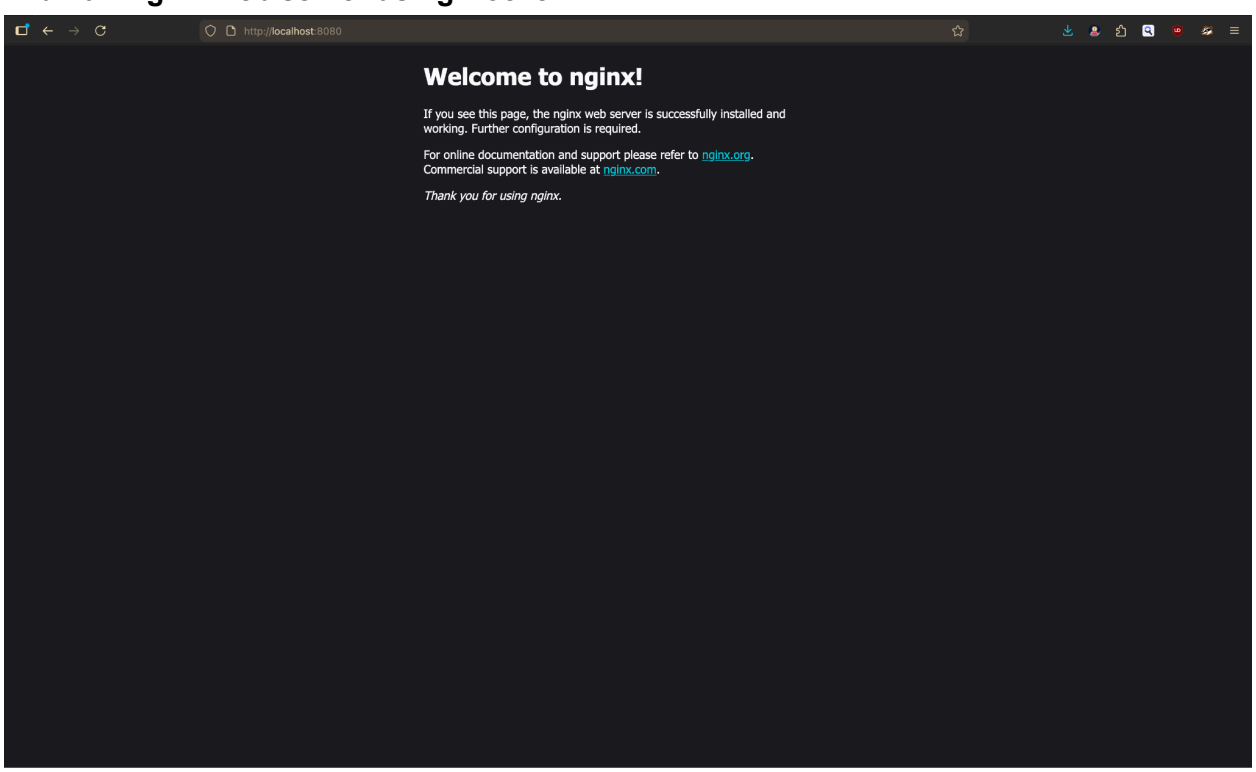
```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                                    zsh

❯ docker build -t my-python-app .
[+] Building 0.2s (8/8) FINISHED                                                                docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                          0.0s
 => => transferring dockerfile: 276B                                                                          0.0s
 => [internal] load metadata for docker.io/library/python:3.11                                                0.0s
 => [internal] load .dockerignore                                                                             0.0s
 => => transferring context: 2B                                                                               0.0s
 => [1/3] FROM docker.io/library/python:3.11@sha256:3b3706a90cb23f04fabb0d255824f9a70ceb46177041898133dd5a35f3a50f0a  0.0s
 => => resolve docker.io/library/python:3.11@sha256:3b3706a90cb23f04fabb0d255824f9a70ceb46177041898133dd5a35f3a50f0a  0.0s
 => [internal] load build context                                                                             0.0s
 => => transferring context: 28B                                                                              0.0s
 => CACHED [2/3] WORKDIR /app                                                                                 0.0s
 => CACHED [3/3] COPY app.py .                                                                                0.0s
 => exporting to image                                                                                        0.0s
 => => exporting layers                                                                                       0.0s
 => => exporting manifest sha256:8123e51ffc2fae64c861759f97dbda0ba697a830120260969daf30236ca8e488             0.0s
 => => exporting config sha256:675a3aa533cf0ad3f6fa63329d7a9e9d19b050261018f170c923064c266922ff               0.0s
 => => exporting attestation manifest sha256:887131ddbda720465724d081614bc4c6a04a8e9a565262c2fe06f5f6645ceb31  0.0s
 => => exporting manifest list sha256:ee5d8faf19f2a794fbe19b290cb74bc0df4643696018842db7b69d581456fb22         0.0s
 => => naming to docker.io/library/my-python-app:latest                                                       0.0s
 => => unpacking to docker.io/library/my-python-app:latest                                                    0.0s
❯ docker run my-python-app
Hello, Docker! This is my first containerized app.

~/syn/edu/uni/sp26/327/327_Docker on main ?2 ····· at 00:10:58
❯
```

5. Task 1: Deploying a web server (Nginx)

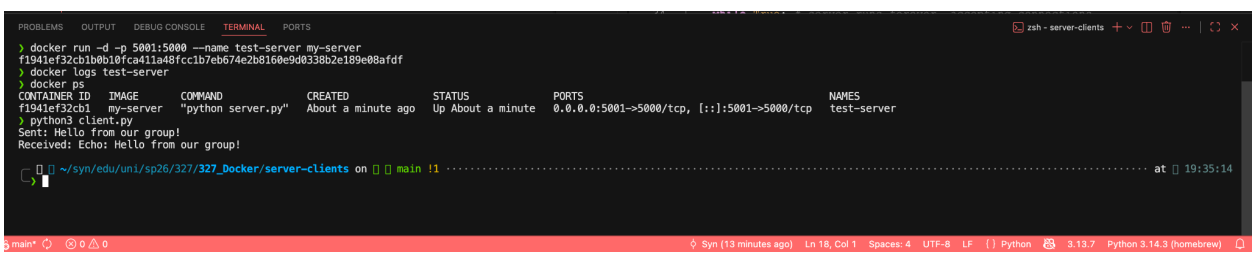
- Run an Nginx web server using Docker



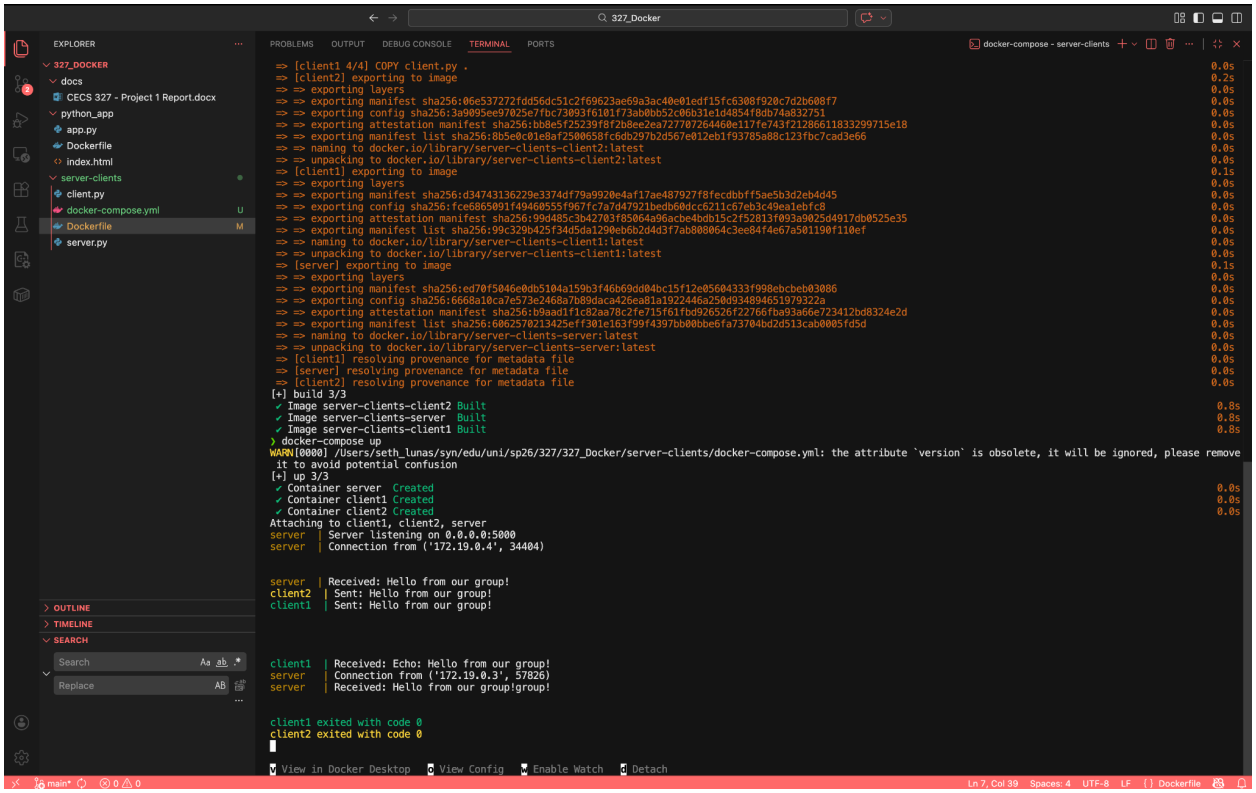
- Modify the default page by mounting volume

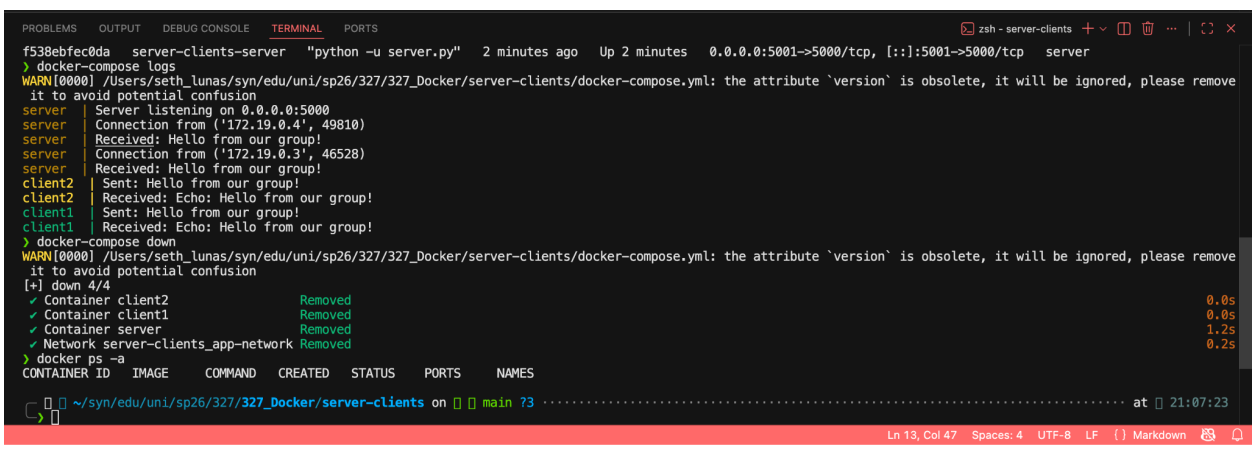
6. Task2: Multi-Container Setup (Server + Clients)

From mac terminal



Connect server and clients using docker compose





Challenges and Solutions

1. Port 5000 conflict - macOS Control Center occupied port 5000. Changed to port 5001 in docker-compose.yml.
2. Missing client.py - Dockerfile only copied server.py. Added `COPY client.py .` to include both files.
3. No logs appearing - Python buffered output. Added `-u` flag to CMD for real-time logging.

Youtube Link: https://youtu.be/4jRV7jLdUFs